INTERNSHIP REPORT

Advanced NLP Multi-Feature Chatbot Suite

Submitted To

ElevanceSkills Technology Private Limited

Mentor-Guided Real-Time Project Internship Program

Submitted By

Alla Dileep Rajesh

B.Tech – Computer Science & Engineering | ANITS

2026

1. Introduction

This report documents the internship at ElevanceSkills Technology, where an AI-powered chatbot suite was developed under the Mentor-Guided Real-Time Project Internship Program. The project involved designing, building, and integrating six progressive NLP and AI features into a unified Streamlit-based chatbot application.

The internship provided hands-on experience in natural language processing, machine learning model training, REST API integration, and full-stack AI application development. Each task built upon the previous one, reflecting a realistic industry development workflow.

2. Background

Conversational AI has grown rapidly, with chatbots deployed across healthcare, education, customer service, and research. This internship focused on building a comprehensive chatbot demonstrating real-world NLP capabilities — from classic machine learning to modern LLM API integration.

Key technologies used: TensorFlow, scikit-learn, FAISS, Sentence Transformers, Google Gemini, Groq Llama 3, arXiv API, langdetect, deep-translator, and the Streamlit web framework.

3. Learning Objectives

- Implement sentiment analysis using machine learning
- Build a retrieval-based QA system on real datasets
- Implement RAG architecture using FAISS vector database
- Integrate real-time data sources (arXiv API)
- Work with multimodal AI (text and image processing)
- Implement multilingual NLP across 5 languages
- Develop production-quality Streamlit web applications
- Apply software engineering best practices in a real project

4. System Architecture

The chatbot is built as a modular multi-page Streamlit application. A central query router dispatches requests to the appropriate NLP module. Each module uses its own session-state namespace to prevent cross-module variable conflicts.

Layer	Component	Technology Stack
Frontend	Unified Chat Interface	Streamlit multi-page app
Routing	Query Router / Sidebar Nav	Python session state
Module 1	Sentiment Analysis	Logistic Regression · SST dataset
Module 2	Medical Q&A	TF-IDF retrieval · MedQuAD 47K+ pairs
Module 3	Dynamic Knowledge Base	FAISS + Sentence Transformers (RAG)
Module 4	arXiv Research Assistant	arXiv API · DistilBART · Plotly

Layer	Component	Technology Stack
Module 5	Multimodal Chatbot	Gemini 1.5-flash + Groq Llama 3.3 70B
Module 6	Multilingual Support	langdetect · deep-translator
Backend AI	LLM Inference Engine	Groq (primary) · Gemini (image tasks)

5. Activities and Tasks

T1 Sentiment Analysis Integration

Trained a Logistic Regression classifier on the Stanford Sentiment Treebank (SST) combined with NLTK Brown Corpus for neutral sentence generation.

- 79% accuracy across Positive, Negative, and Neutral classes
- Color-coded real-time sentiment display with empathetic responses
- 50% confidence threshold prevents misclassification of short messages

T2 Medical Q&A; Chatbot (MedQuAD)

Parsed 15,000+ XML files from the MedQuAD dataset containing 47,457 NIH medical QA pairs for TF-IDF-based retrieval.

- Basic medical entity recognition for symptoms, diseases, and treatments
- Score boosting/penalization resolves rare-disease false positives
- XML caching for fast load · hardcoded fallbacks for common symptom queries

T3 Dynamic Knowledge Base (RAG)

Built a Retrieval-Augmented Generation system using FAISS vector database and Sentence Transformers for document-grounded responses.

- Supports TXT, PDF, and DOCX document uploads; chunked and embedded on upload
- Distance threshold (MAX_DISTANCE = 1.2) prevents irrelevant matches
- Tested with an Anatomy & Physiology PDF for domain accuracy

T4 Domain Expert Chatbot (arXiv)

Integrated the arXiv API for real-time computer science research paper search with extractive summarization and interactive visualizations.

- Keyword-frequency summarization with DistilBART fallback
- Follow-up question handling via active paper tracking
- Interactive Plotly charts: concept networks, timelines, category distribution

T5 Multimodal Chatbot

Integrated Google Gemini 1.5-flash for image understanding. Due to regional quota limits, Groq Llama 3.3 70B was approved by the mentor as the primary text engine.

- Image queries routed to Gemini Vision · text queries to Groq Llama 3
- PIL-based demo fallback for Gemini quota exhaustion scenarios
- Groq fallback documented and approved within internship scope

T6 Multilingual Support

Implemented automatic language detection for English, Telugu, Hindi, Spanish, and French using langdetect and deep-translator.

- Input translated to English for processing; response translated back
- Culturally appropriate sentiment prefixes in each language
- Replaced incompatible googletrans with deep-translator for Python 3.10+

6. Performance Metrics

Module	Key Metric	Value / Status
Sentiment Analysis	Classification Accuracy	79%
Medical Q&A	Dataset Scale	47,457 QA pairs · 15,000+ XML files
Dynamic Knowledge Base	Retrieval Method	FAISS + Sentence Transformers (RAG)
arXiv Research	Data Source	Live arXiv API (real-time results)
Multimodal Chatbot	Models Used	Gemini 1.5-flash + Groq Llama 3.3 70B
Multilingual Support	Languages Supported	5 (EN · TE · HI · ES · FR)
Overall Application	Modules Completed	6 / 6 ✓

7. Project Structure

```
NLP_CHATBOT/
chatbot_pages/ # Per-task Streamlit page modules
services/ # API wrappers: Gemini, Groq, arXiv
vector_db/ # FAISS index and document store
data/ # MedQuAD cache, SST training data
screenshots/ # UI evidence screenshots
```

```
main_chatbot.py # Entry point — streamlit run main_chatbot.py
requirements.txt # Pinned dependency versions
README.md
Project_Report.pdf
```

8. Skills and Competencies

Technical Skills Gained

- ML model training (Logistic Regression, Neural Networks)
- NLP: tokenization, lemmatization, TF-IDF, embeddings
- FAISS vector database design and RAG architecture
- REST API integration (arXiv, Gemini, Groq)
- Streamlit multi-page web application development
- Python best practices: modular code, caching, error handling
- Working with large-scale datasets (MedQuAD, SST, arXiv)

Soft Skills Developed

- Independent problem solving and self-directed research
- Debugging complex multi-dependency Python environments
- Technical documentation and structured report writing
- Managing project scope and task prioritisation
- Version control workflow (Git)
- Iterative development and mentor communication

9. Challenges and Solutions

Protobuf Conflicts	**Problem:** Installing google-generativeai caused protobuf version conflicts with TensorFlow.	**Solution:** Managed package versions carefully; used --no-deps to prevent dependency resolution from breaking existing packages.
Gemini API Quota	**Problem:** Gemini free tier has severely restricted quotas for India-region users.	**Solution:** Reported to mentor; received official approval to use Groq Llama 3.3 70B as primary text engine while retaining Gemini for image tasks.
MedQuAD Retrieval Accuracy	**Problem:** Initial TF-IDF retrieval returned rare-disease results for common everyday queries.	**Solution:** Implemented similarity score boosting/penalization by disease type; added hardcoded fallbacks for common symptom queries.

Translation Library Incompatibility	**Problem:** The googletrans library was incompatible with newer Python/httpcore versions.	**Solution:** Migrated to deep-translator, which is actively maintained and fully compatible with Python 3.10+.
Knowledge Base Over-Matching	**Problem:** FAISS returned document content in response to unrelated casual queries.	**Solution:** Introduced a distance threshold (MAX_DISTANCE = 1.2) to filter out weak vector matches and suppress irrelevant responses.
arXiv SSL / Rate-Limit Errors	**Problem:** arXiv API calls failed intermittently due to SSL certificate errors and rate limiting on Windows.	**Solution:** Implemented exponential backoff retry logic and verify=False for SSL; ensured stable retrieval in the development environment.

10. Industry Applications

Healthcare	Medical virtual assistants and symptom checkers powered by QA retrieval
Education	AI tutoring systems with multilingual and document-aware responses
Customer Support	Intelligent ticket routing with sentiment-aware empathetic responses
Research	Academic literature discovery via real-time arXiv search and summarization
Enterprise	Internal knowledge management using RAG over proprietary documents
Accessibility	Multilingual interfaces enabling non-English speaking users

11. Future Scope

- Voice interaction using Whisper / Web Speech API
- Cloud deployment on AWS or Azure with containerised microservices
- Fine-tuning domain-specific LLMs on medical and research corpora
- Integration with Electronic Health Records (EHR) systems
- Real-time web search augmentation using Tavily or SerpAPI
- Mobile deployment via React Native or Flutter wrapper
- Feedback loop for continuous model improvement via user ratings

12. Outcomes and Impact

The internship resulted in a fully functional AI chatbot suite. All six modules were completed and integrated into a single production-quality Streamlit application demonstrating end-to-end NLP and AI development competency.

Module	Description	Status
Sentiment Analysis	3-class classifier with empathetic responses	✓ Completed
Medical Q&A (MedQuAD)	47K+ NIH QA pairs with TF-IDF retrieval	✓ Completed
Dynamic Knowledge Base	FAISS RAG system with document upload	✓ Completed
arXiv Research Assistant	Live paper search with summaries & charts	✓ Completed
Multimodal Chatbot	Image understanding (Gemini) + text (Groq)	✓ Completed
Multilingual Support	5-language auto-detect and translate	✓ Completed

The project reflects real-world software development workflows: iterative development, multi-library dependency management, API integration, fallback design, and polished UI delivery — all within a mentor-guided internship timeline.

13. Project Resources

Resource	Link / Location
GitHub Repository	https://github.com/DileepRajesh-2006/NLP-MultiFeature-Chatbot *Live repository containing source code, screenshots, datasets, and documentation.*
Dataset (Google Drive)	https://drive.google.com/drive/folders/1rRzCOhllsi7gtyjVQY5u36-ZFUczwZSc
Trained Model Files	chatbot_model.keras · sentiment_model.pkl · tfidf_vectorizer.pkl
Application Entry	main_chatbot.py (run: streamlit run main_chatbot.py)

14. Conclusion

This internship provided valuable hands-on experience building a comprehensive AI chatbot system from scratch. Each task progressively added new capabilities, demonstrating how modern NLP and AI technologies combine to create powerful, real-world conversational applications.

The challenges encountered — from dependency conflicts to regional API quota limitations — provided practical problem-solving experience that extends well beyond theoretical knowledge. The project successfully integrates six distinct AI capabilities into a single cohesive platform.

The skills and experience gained provide a strong foundation for future work in AI, NLP, and software development — demonstrating the ability to independently research, implement, debug, and deliver real-world AI applications within a professional internship setting.